

Is software “green”? Application development environments and energy efficiency in open source applications

Eugenio Capra^{a,*}, Chiara Francalanci^a, Sandra A. Slaughter^b

^a Dipartimento di Elettronica e Informazione, Politecnico di Milano, via Ponzio 34/5, I-20133, Milano, Italy

^b College of Management, Georgia Institute of Technology, 800 West Peachtree Street NW, Atlanta, GA 30308, United States

ARTICLE INFO

Article history:

Received 16 January 2011

Received in revised form 25 May 2011

Accepted 2 July 2011

Available online 26 August 2011

Keywords:

Green IT

Software energy efficiency

Software development application environment

ABSTRACT

Context: The energy efficiency of IT systems, also referred to as Green IT, is attracting more and more attention. While several researchers have focused on the energy efficiency of hardware and embedded systems, the role of application software in IT energy consumption still needs investigation.

Objective: This paper aims to define a methodology for measuring software energy efficiency and to understand the consequences of abstraction layers and application development environments for the energy efficiency of software applications.

Method: We first develop a measure of energy efficiency that is appropriate for software applications. We then examine how the use of application development environments relates to this measure of energy efficiency for a sample of 63 open source software applications.

Results: Our findings indicate that a greater use of application development environments – specifically, frameworks and external libraries – is more detrimental in terms of energy efficiency for larger applications than for smaller applications. We also find that different functional application types have distinctly different levels of energy efficiency, with text and image editing and gaming applications being the most energy inefficient due to their intense use of the processor.

Conclusion: We conclude that different designs can have a significant impact on the energy efficiency of software applications. We have related the use of software application development environments to software energy efficiency suggesting that there may be a trade-off between development efficiency and energy efficiency. We propose new research to further investigate this topic.

© 2011 Elsevier B.V. All rights reserved.

1. Introduction

There is clear evidence that energy costs are becoming increasingly relevant. Information technology (IT) has a predominant role in reducing energy consumption, both as a tool to monitor and optimize the energy efficiency of any production process, and as a target of energy efficiency initiatives. The term *green IT* refers to both of these aspects of the relationship between IT and energy efficiency. This paper focuses on the energy efficiency of IT and, specifically, on the energy efficiency of software. Accordingly, we use the term green IT to indicate the issues related to the energy efficient design of IT.

Most research on green IT focuses on the energy efficiency of hardware [5,34]. As power is physically supplied to machines, energy costs are naturally associated with hardware and are most visible in the data center where they add up to significant dollar figures. Currently, half of the total cost of ownership (TCO) of a

midrange server, e.g. a blade server, is due to the energy costs of power and cooling [23,27]. Large data centers require a constantly growing amount of energy that will soon become difficult to supply in a single geographical location with current energy infrastructures. As a result, companies with large data centers are starting to inquire whether the energy efficiency problem should be tackled from a software perspective, with the goal of controlling the growing trend of computing capacity requirements [6]. In particular, recent research indicates that end-user application software, such as management information systems (MIS), have a significant impact on IT energy consumption [12]. Experimental results show that the application layer can increase the total consumption of a server up to 72% with respect to the system in idle state. Similar experiments have shown that different MIS applications that satisfy the same functional requirements and run on the same hardware and operating systems have significantly different consumption (up to 145%). This suggests that factors such as software development (or maintenance) approaches may have an impact on IT energy consumption.

According to Stanford [42], for each watt spent to power a server's CPU, 28 watts must be fed to the data center hosting the

* Corresponding author. Tel.: +39 02 2399 4014; fax: +39 02 700502112.

E-mail addresses: capra@elet.polimi.it (E. Capra), francala@elet.polimi.it (C. Francalanci), Sandra.Slaughter@mgt.gatech.edu (S.A. Slaughter).

server, due to energy overhead and inefficiencies. Although software does not directly consume energy, it deeply affects the consumption of hardware equipment, as it indirectly guides its functioning. All the infrastructural layers in a data center amplify the energy consumption induced by software. This suggests that a small reduction of CPU usage obtained by running greener software can provide significant energy savings: if fewer elementary operations are required to satisfy a user functional requirement all the above layers (bus, storage, cooling, etc.) are likely to work less and thus require less energy (see [11]). Even more interesting, under some conditions (namely, an optimized use of servers through virtualization, or the use of a mainframe system, or the application of the ASP approach), a reduction of CPU usage involves a corresponding reduction of the TCO of the data center in an almost linear relationship [3]. Since the annual energy bill for very large data centers is in the millions of dollars, investing in green software design may prove economically efficient even for a single company. If the investment is made in the green redesign of packaged software with installations in multiple companies, benefits could even have a socio-economic impact.

Application development environments have become the foundation of modern software development approaches. An application development environment is the provision of hardware and software tools to develop an application. An example tool is a framework such as HIBERNATE which allows programmers to access a database without having knowledge of the underlying data schema. Application development environments have encouraged the simplification of programming tasks through higher level programming languages, frameworks, and libraries [18]. More generally, we include in the category of application development environments all the abstraction layers that embed knowledge and simplify the job of developers. The main goal of these environments has been to reduce development time and cost by simplifying programming tasks and streamlining the software production process itself. The premise has been that both speed and cost goals can be achieved if programmers operate within a development environment that provides them with a number of reusable modules and high level constructs. But can application development environments also be beneficial to software energy efficiency?

Generally speaking, the software tools included within current application development libraries are designed for both development process performance and efficiency. However, if programmers are not adequately instructed to be aware of energy consumption issues when using these tools, they could easily write energy inefficient code. It is a common experience that newer releases of software are more resource crunching, even in personal computing environments. Traditionally, this was not considered a problem, since more hardware resources could be added at a relatively low cost. However, this is no longer the case as the cost of energy to run the hardware has risen dramatically.

Therefore, this paper has two objectives:

- (1) Develop a measure of energy efficiency for software applications. While most attention has been placed on improving energy efficiency for computing hardware, we argue that considerable gains may be achieved by also considering how to make software applications more energy efficient. A first step is to develop a measure of energy efficiency that is relevant for software applications, and we do that in this paper.
- (2) Investigate the relationship between the use of application development environments and software energy efficiency. Application development environments are commonplace today and have helped to streamline programming tasks. However, it is not clear whether these approaches will also result in higher software energy efficiency.

We measure the energy efficiency of a sample of 63 open source applications. Our focus is on application level software, which is utterly different from embedded systems and low-level software, as will be discussed in Section 2.1. We then assess the extent to which these applications have been developed using application development environments. The use of the advanced features of modern application development environments is measured by means of a new indicator we have created, called *framework entropy*, which indicates the level of abstraction adopted to develop a given fragment of code. This is related to the extent to which libraries and higher level constructs are used. This concept goes beyond the simple reuse of code. Framework and external libraries provide code that per definition should be general and encompass all the possible situations. This code may be inefficient when compared with code developed ad hoc. We relate measures of framework entropy to energy efficiency for our sample of open source software applications.

The paper is organized as follows: Section 2 discusses the state of the art related to the research objectives proposed above, Section 3 proposes the research hypotheses of this work, Section 4 describes how variables have been operationalized and, in particular, how we measured the energy efficiency of a software application. Section 5 presents the statistical methodology and results of the analyses, which are finally discussed in Section 6. Section 7 discusses contributions, limitations, and directions for future work.

2. Related work

This section reviews the main studies that have addressed IT energy efficiency. Section 2.1 discusses the streams of literature that have tackled energy efficiency from a software perspective. Section 2.2 focuses on the impact of application development environments on energy efficiency.

2.1. Software energy efficiency

Over the years, hardware energy efficiency has significantly improved, with particularly high gains in the energy efficiency of mobile devices as a response to battery autonomy issues. Over the past 30 years, the value of MIPS/W (Million of Instructions per Second per Watt) of mainframe systems has increased by a factor of 28, which represents an improvement much higher than that achieved by production machines in other industrial sectors, such as steel production or automotive [1]. Nevertheless, whereas hardware has been constantly improved to be energy efficient, software has not. The software development life cycle and related development tools and methodologies rarely, if ever, consider energy efficiency as an objective. The availability of increasingly efficient and cheaper hardware components has led designers to neglect the energy efficiency of software, which remains largely unexplored. Some specific research studies of software energy efficiency have been conducted in the field of mobile computing or embedded systems (see for example [24]), but these studies mainly focus on infrastructural and low level software rather than on application software. The current literature does not even provide software energy efficiency metrics. Not surprisingly, the over 50 ISO software quality parameters ([22]) do not include energy efficiency.

It must be observed that previous research on *embedded software* has focused on improving software performance to reduce overall CPU usage and, thus, energy consumption [5]. The literature on the energy efficiency of embedded systems is vast and provides methodologies both to estimate energy efficiency and to design for energy efficiency. Chatzigeorgiou and Stephanides [13], and Fornaciari et al. [14] propose methodologies to estimate software energy efficiency. For example, Albers and Fujiwara [2] study

scheduling problems in computer devices operated with batteries with the aim of minimizing energy consumption without losing in quality of service. Sivasubramaniam et al. [41] investigate development techniques to design energy efficient software for embedded systems.

Compared to application software, embedded software has a greater possibility to influence hardware settings, as well as a very limited functional complexity. This allows development with low-level languages and code optimization based on a direct control over elementary operations. In contrast, application software development uses higher-level languages, libraries, and frameworks that are meant to reduce the complexity of development tasks, but change the nature of the resulting applications, transforming them into complex software ecosystems for which time performance and energy consumption may become separate and often conflicting issues. Application software involves a number of additional design choices at the architectural design level that may have an impact on energy consumption, as shown by [36,37], who developed a framework to estimate the energy consumption of software systems implemented in Java at construction-time. However, the impact of architectural constructs on energy efficiency still needs to be investigated in order to provide developer teams with methodologies and guidelines for green software development. In a previous work [12], we empirically analyzed the energy consumption induced by comparable MIS applications (namely, ERPs, CRMs, and DBMS) and found out that: (i) not only the infrastructural layers, but also the MIS application layer does impact energy consumption (up to 70%); (ii) different MIS applications satisfying the same functional requirements consume significantly different amounts of energy (differences up to 145%); and (iii) in some scenarios energy efficiency cannot be increased simply by improving time performance.

A common misconception is to equate energy efficiency to software performance. The rationale of this erroneous belief is that increasing software performance involves a reduction of CPU usage and, hence, energy consumption. If this is true, energy efficiency could be obtained by capitalizing on the vast literature on software performance (cf. for example [45,40,28]). However, this same literature explains that if performance is measured from a user's perspective, i.e. in terms of response time, improving performance may involve an *increase* as opposed to a reduction of CPU usage. Furthermore, reducing CPU usage may not translate into energy savings. In order to understand why time performance and energy consumption can be distinct goals, let us consider a large and functionally complex application, such as an Enterprise Resource Planning (ERP). To cope with complexity, designers sometimes disentangle different application modules by coordinating them through the database. This approach to the architectural design of software increases the number of accesses to the database and, if caching is not used properly, makes applications slower. However, it may not increase energy consumption as CPU usage can be reduced, while greater disk usage does not increase energy consumption since idle and busy disks have approximately the same power requirements. The power requirements of disks are in fact mostly related to spinning, irrespective of actual data transfers. In a previous paper, we have provided empirical evidence showing how faster applications may also involve higher energy consumption [12].

Moreover, using memory effectively to reduce the need for garbage collection does not decrease the response time of a program to a short lived test, but it improves the efficiency of the runtime garbage collector of the software framework which, in turn, may reduce the overall usage of CPU (cf. [32]).

These considerations suggest that, at a closer look, energy efficiency and performance represent separate and possibly conflicting issues. A significant literature has focused on time

performance [39,45,40,28,25], but to the best of our knowledge, there are no previous studies relating time performance to the energy efficiency of application software.

Several studies have been conducted to improve the energy efficiency of software at the compiler level. The compiler can affect power consumption by applying program transformations [46], or by driving dynamic voltage scaling and adaptive body biasing [20], but these techniques are more appropriate for battery operated devices than for servers [6]. In the context of complex applications relying on multiple servers and clients communicating through a network, causes for high energy consumption include sloppy programming techniques, excessive layering (deep inheritance trees leading to higher method invocation costs), code fragmentation (excessively small classes, small code objects inhibiting aggressive optimization), and over-design (e.g., using databases to hold static configuration data). The compiler cannot easily compensate for these kinds of issues – thus, further investigations are needed to identify more targeted design methodologies.

2.2. Application development environments

In the early days of programming, there were low level programming languages such as assembler, and few tools a programmer could leverage (other than the programmer's own ability) to create code. Thus, only skilled developers were able to be highly efficient in coding applications. This created a problem for companies, as these highly skilled individuals were scarce, costly, and could leave the company, taking their knowledge and skills with them. Thus, application development needed to be simplified and streamlined so that even less skilled developers could efficiently create application programs. At the same time, software has been becoming more and more complex and challenging to create, even for highly skilled developers, given the increasing demands for greater, more novel and complicated functionality [30,7]. All of these factors have led to the rise of application development environments which provide hardware and software tools to aid the development process, such as frameworks and libraries of code modules that can be re-used. More generally, we included in the category of application development environments all the abstraction layers of high level software applications.

The traditional software engineering literature concurs in associating these environments with better software development efficiency. For example, Steffen and Narayan [43] have provided empirical evidence indicating that disciplined development methodologies based on the use of application development environments increase development efficiency. Marttiin et al. [31] provide a framework to study the usability of CASE tools and analyze their impact on the effectiveness of development.

Several studies have analyzed the best practices of componentization and reuse from a process efficiency perspective (see, for example, [15,17,16]) but, to the best of our knowledge, no empirical evidence has been provided to show the impact of application development environments on the energy efficiency of software applications. This is not surprising as the software development life cycle and related process management methodologies do not consider this parameter (see [21]).

3. Research hypotheses

Generally, our expectation is that the use of application development environments will be detrimental to software energy efficiency. This is for two reasons. First, application development environments simplify development by protecting programmers from having to understand the underlying complexities of the technology. The aids provided by these environments can help a pro-

grammer to be more efficient in coding but may cause extra work for the processor when a program is executed. As an example, let us consider the execution of a query on a database through a basic SQL interpreter versus its execution through Microsoft Access. Microsoft Access supports the developers in the programming process, and most of the queries can be formulated without knowing the SQL keywords by means of simple visual and CASE-like interfaces. An intermediate software layer between the user interface and the query interpreter will then translate the requests of the developers into SQL code. Database systems run on Access are easy to use and do not require highly specific skills if compared to database creation and querying through pure SQL. However, Access does introduce additional layers that per se consume additional power. Moreover, the SQL code that is finally executed is generated through a software layer that interprets the requests of the developer, and is not optimized. It is common experience that code written by an automatic interpreter is less efficient than code written by an experienced developer (see also <http://databases.aspfqa.com/database/what-are-the-limitations-of-ms-access.html> for a critical discussion of performance limitations of Microsoft Access). In general, additional software layers often introduce inefficiencies that affect CPU-time.

Second, application development environments are designed to be general and enable the reuse of portions of code, packaged as libraries, standard routines, and framework functionalities [26,44]. It is likely that general purpose components will have many extraneous features that are less relevant for a particular task, and that loading and executing these components will make for extra work by the processor when the program is executed. For example, an ERP application may rely on a framework to access the database, e.g., the framework HIBERNATE, which provides ORM (Object-Relational Mapping) functionalities to manage data. This framework includes several classes (totaling 10 Mbyte in size). These classes are easily usable and maintainable by non-expert developers, but they introduce an overhead during the execution. In fact, in order to be general, the framework needs to include all the potential operations and data structures, even though not applicable to the specific ERP module. Additional internal variable instantiations, data conversions and controls are executed, even though not strictly necessary for the specific application. All these operations constitute an overhead for the processor executing the application. As an alternative, an ERP may include one special purpose written class (requiring a few Kbyte) with the SQL instructions to access the database. This approach may require a more skilled developer and may be less maintainable, but it would reduce energy consumption, as the program needs just one module to execute and that module has only the code needed to accomplish the particular function.

Thus, our expectation is that:

H1. A higher use of application development environments has a detrimental effect on software energy efficiency.

As applications become larger and more complex, we expect that the increased use of application development environments may be even more detrimental, with respect to energy efficiency. The growth of a complex application occurs by progressively embedding initial modules inside other larger modules. As a result, the number of layers that must be crossed to execute a single operation can become unnecessarily high as an application increases in size. Larger and more complex applications require special treatment to counter the layering effect induced by componentization. Consistent with these considerations, our second hypothesis poses that:

H2. The detrimental effect of using application development environments on energy efficiency is more pronounced for larger than for smaller applications.

4. Variable definition and operationalization

In this section, we provide the formal definition and operationalization of the variables used in our study. Energy efficiency is measured by means of a hardware experimental kit, and code based metrics are measured by means of a code analyzer, both built ad hoc for this study.

4.1. Energy efficiency

Energy efficiency measures the energy consumed by a given hardware system while responding to a set of requests of an application divided by the total number of requests. Measuring the energy consumed to respond to one request is beyond the sensitivity threshold of commercial sensors. Moreover, in an enterprise application context it is difficult to define the concept of request, given that each application can generate a variety of requests. To overcome these difficulties, we define *specific energy* as the energy absorbed by a system running an application executing a given functional workload involving multiple requests compared to the average energy absorbed by applications belonging to the same functional area (i.e. with the same set of functionalities) to execute the same workload.

The operating definition of specific energy is obtained as follows. Given an application, say i , belonging to a functional area, say A , specific energy SE_i is defined as:

$$SE_i = \frac{Ed_i - \overline{Ed}_A}{\overline{Ed}_A}, \quad (1)$$

where Ed_i is the difference between the power absorbed by the system running application i and the power absorbed by the system in idle, integrated over the time required to complete the workload; $\overline{Ed}_A$ is the mean value of Ed of applications in functional area A .

The lower the *specific energy* required to execute a set of benchmark workloads, the higher the *energy efficiency*. Accordingly, we define the *energy efficiency* of an application i as:

$$EE_i = 1 - SE_{iNORM} \quad (2)$$

where SE_{iNORM} is the value of SE_i normalized to values between 0 and 1 over the sample of applications considered within the same functional area.

Since specific energy is normalized, it can be compared across different functional areas. The energy efficiency of different applications can then be assessed by comparing their specific energy calculated for the same workload.

We measured energy efficiency by building: (i) a tool to generate workloads for different categories of applications and (ii) a hardware kit measuring the energy absorbed by the system.

A test workload has been defined for each category of applications. For example, the workflow of ERPs has been generated by executing the flow of operations to create a business partner, a product, and an order of the product to the partner. We have implemented a Java tool, called *Workload Simulator*, that can record a given flow of operations and execute it a given number of times for a given number of simultaneous users. While recording a workload, the *Workload Simulator* eliminates the thinking times between subsequent operations to allow comparisons across different applications. As shown in Fig. 1, the *Workload Simulator* executes and synchronizes the benchmark workloads on a number of different clients. The clients send requests to a server machine executing an application of the corresponding functional area (e.g. an ERP). The power measured is that consumed by the server machine.

The Virtual Instrument Machine is connected to the hardware measurement kit and runs the Labview Virtual Instrument

described below in order to monitor and store the power consumption values of the Server Machine. It is important to note that all the tools needed for controlling the simulation and measuring power consumption do not run on the Server Machine in order not to interfere with the experiment. Table 1 reports the configuration of the Server Machine used for all simulations.

The current absorbed by the Server Machine is measured by means of an ammeter clamp. Ammeter clamps have a Hall current sensor inside and allow non intrusive measures.

The analog signal acquired by the ammeter clamp is processed by a NI USB-6210 DAQ (Data Acquisition Board) that is interfaced via USB with the Virtual Instrument Machine. Data acquisition is performed by a software tool developed ad hoc for this study using LabVIEW (www.ni.com/labview/). The tool acquires and stores samples of current energy consumption every 4 ms (i.e., with a sampling frequency of 250 MHz) and computes the total energy consumed over time by interpolating current consumption samples.

4.2. Framework entropy

The testing of the proposed hypotheses requires a measure of the use of libraries and high-level framework constructs within a software application. Note that a simple count of external libraries used could be misleading. In particular, including one library and using it multiple times is different from including multiple libraries and using each of them once. Both the quantity and the variety of high-level constructs used in a given application are relevant for our purposes. In order to obtain a more precise measure, we developed a new metric, called *framework entropy*.

Framework entropy draws from the theoretical discussion presented by Capra and Merlo [11]. It stems from the observation that different programmers can use the framework entropy of a development language to different degrees. *Shannon's entropy* [38] has been defined within communication theory to measure the regularity with which different symbols occur within a text. Formally, Shannon's entropy is defined as follows:

$$H = - \sum_{x \in S} p(x) \log_2 p(x), \quad (3)$$

where S is the set of all symbols x of a language, and $p(x)$ is the probability of occurrence of symbol x .

Shannon's entropy is at a minimum if the text uses a limited set of symbols evenly, while it is at a maximum if it uses a wider set of symbols, but with irregular frequency. Shannon's entropy can be applied to the code with reference to a given programming environment by considering as symbols the keywords of the language, names of methods, objects, user-defined variables, and functions, imported constants and functions (from frameworks and external libraries), operators and mathematical expressions.

We refer to this application of Shannon's entropy as *ENTROPY*. Framework entropy denotes the programming style of different developers. If the developer uses all the keywords and constructs provided by the environment, entropy will be high. On the other hand, if the developer makes little use of external libraries and high level constructs, while implementing functions using the base keywords of the language, the programming style will be less entropic. In summary, the higher the framework entropy metric, the higher the use of external libraries and high level constructs.

For example, let us consider two similar implementations of an ERP module, say A and B, that execute a flow of operations and at a certain point sort a list of items. Module A implements a Bubble Sort algorithm using the basic keywords of the language (for, if, etc.). Module B imports the BubbleSort function from an external library. According to our definition, Application B has greater framework entropy than Application A.

The metric of framework entropy has been evaluated by means of an ad hoc tool, developed in Java, which parses the code of Java applications. We manually inspected the results for a few applications in our sample and verified that our interpretation of the metric is correct.

4.3. Size

The size of applications has been measured as: (1) the binary length of the bytecode and (2) the total number of methods. These metrics are automatically extracted by the ad hoc code analyzer tool that we used to measure framework entropy.

4.4. Age

We controlled for the age of each project in our regressions. The age is measured as the number of years elapsed from the registration of the project on SourceForge.net.

4.5. Functional types

As we will discuss in Section 5.1, our sample includes different types of applications. We controlled for functional types by means of binary variables. Each of these variables represents a particular functional type and assumes a value of 1 for all the applications belonging to that type, and value 0 for all the other applications.

5. Methodology and results

The next two sections provide a description of the data sample (Section 5.1) and statistical approach (Section 5.2). Results from statistical analysis and hypothesis testing are then reported in Section 5.3.

5.1. Data sample

Hypotheses are verified on a cross-section of 63 open source applications belonging to different functional areas and selected from the *SourceForge.net* software repository.

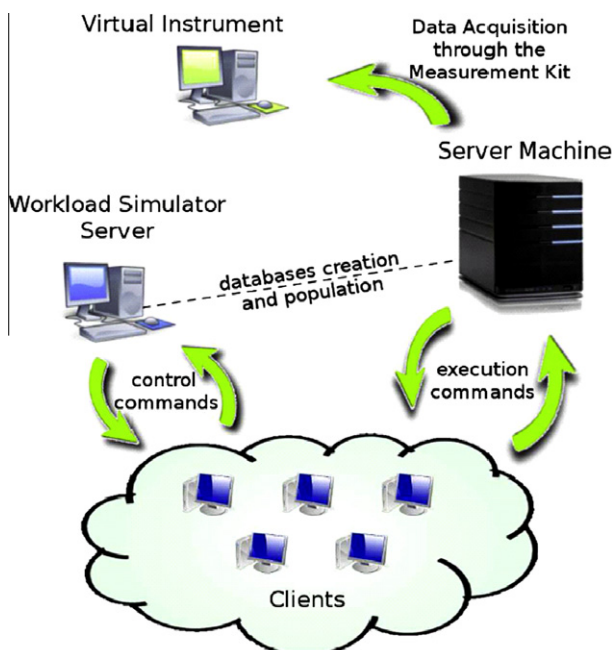


Fig. 1. System architecture for workload simulation.

Table 1
Server machine configuration.

Parameter	Value
Processor	2x Intel Xeon 2.40 GHz
Cores	2 per processor
Internal data cache	2 × 8 kb
On-board cache	2 × 512 kb
Motherboard	Asus PR-DLS
Total memory	1 GB DIMM
Memory Bus Speed	4 × 100 MHz (400 MHz)
Chipset	Server Works CMIC-LE
Storage Device	68 GB SCSI Hard Disk
Operating system	Microsoft Windows 2003 Server Enterprise Editions and Linux CentOS

We selected the SourceForge software repository because: (i) it is one of the most widely known and used, and (ii) it provides useful project metadata to select applications.

Since mining data from on-line communities can lead to controversial results because of the varying quality of available data [19], we carefully selected the projects for our study by applying the following criteria:

- *Project maturity*: beta status or higher. Less mature projects were excluded because of their instability and low significance.
- *Language*: Java, in order to make metrics comparable.
- *Number of active contributors*: at least 3 active contributors.

The Java language has been chosen since (i) it is an object-oriented language and many metrics apply to object-oriented applications, and (ii) it is one of the most common languages, especially in the open source context.

The evaluation of energy efficiency requires applications that are comparable from a functional point of view, even if they have different structures. From a functional point of view, two applications are considered comparable if they offer the same functionalities. In order to make our analyses more accurate, we considered two applications comparable only if they are written in the same programming language, deployed on the same software stack and configured with the same parameters (e.g., an ERP system running on the same database, configured with the same indexing, caching, and buffering settings). An application is considered more energy efficient than another application if it responds to the same request with lower energy consumption on the same hardware. Therefore, all tests have been performed on the same hardware configuration (see Table 1).

To gauge the extent to which energy efficiency differs across functional areas, our sample includes different functional types, and we evaluated energy efficiency in sub-samples of comparable applications within each function. We chose these functional areas according to their diffusion and their significance in corporate contexts. We included games in the sample because they are good example of computation intensive applications and are easy to find as open source versions.

Table 2 describes how the applications in our sample have been categorized in functional types.

Table 3 provides summary descriptive statistics of the sample of applications. As ERPs are very large applications compared to the

others in the sample, we only considered the modules stressed by our workload tests.

As hypothesis H2 suggests an interaction effect of framework entropy with size, we divided our sample of applications in two clusters according to size. Table 3 reports the descriptive statistics of the metrics of size measured by our tool. We adopted the mean values of these variables as thresholds to discriminate between the two clusters.

Accordingly, cluster C0 contains all the applications with binary length >61,122 and number of methods >1610 (large applications), whereas cluster C1 contains all other applications (small applications). As a result, cluster C0 includes 39 applications, cluster C1 includes 24 applications.

We assigned each application in the sample a variable *SMALL* that equates to 1 if the application belongs to cluster C1 (small applications) and 0 if it belongs to cluster C0 (large applications).

Table 4 reports the pairwise correlations between the variables in the regression model. Pearson correlation indices are used for *EE*, *ENTROPY*, and *AGE*, which are continuous variables, while Spearman's rho is indicated for correlations with all the other non-continuous variables. As can be seen in Table 4, the pair-wise correlations between the variables in our analysis are modest, suggesting little problems with collinearity.

Table 5 reports the means and standard deviations of the variables in the regression model.

5.2. Statistical approach

In Section 3 we proposed a theory on what determines software energy efficiency and posed hypotheses to verify our model. In this section, we define a regression model based on our theory. The main difference between regression and correlation is that the former assumes an interpretation framework (i.e., a theory about the interaction of variables) to define meaningful causal relationships.

Framework entropy is a property directly associated with the development process, whereas energy efficiency is an outcome of the execution of an application. While it is reasonable to hypothesize that a property of the application or its development process impacts its execution, the opposite causal implication does not seem meaningful.

We tested our hypotheses using the following multiple regression model:

$$\begin{aligned}
 EE = & b_1 \cdot ENTROPY + b_2 \cdot SMALL + b_3 \cdot SMALL \cdot ENTROPY \\
 & + b_4 \cdot AGE + b_5 \cdot FTP + b_6 \cdot CALC + b_7 \cdot CALENDAR \\
 & + b_8 \cdot IMAGE + b_9 \cdot TEXT + b_{10} \cdot WEB + b_{11} \cdot ERP \\
 & + e
 \end{aligned} \quad (4)$$

AGE, *FTP*, *CALC*, *CALENDAR*, *IMAGE*, *TEXT*, *WEB*, *ERP*, and *GAME* are control variables.

Regression hypotheses are supported by findings if (i) the sign of regression weights is consistent with the direction of the hypothesized causal relationships, and (ii) regression weights are statistically significant, with *p*-values less than or equal to 5%. In particular, the regression weights of the terms multiplied by *SMALL* indicate how the relationships change when the cluster of small applications is considered. As hypothesis H2 suggests an interaction effect between framework entropy and size, a correct interpretation of the data also requires us to differentiate expression (4) with respect to *ENTROPY* and to study how *EE* varies as a function of *SMALL* and *ENTROPY*.

Fig. 2 provides an overview of the statistical model used to test our hypotheses. The squares in the diagram represent the variables in the statistic model, and the arrows represent the causal relationships that will be tested.

Table 2
Functional types of applications in the sample.

Functional type variable	Application category	Number of applications in the sample
ERP	ERP	2
FTP	Ftp servers	4
	Ftp clients	5
CALC	Calculators	6
	Spreadsheets	5
CALENDAR	Calendars	5
IMAGE	Image editors	3
	PDF mergers	2
TEXT	Text editors	4
WEB	Mail servers	3
	Web browsers	2
GAME	Arkanoid	2
	Backgammon	3
	Mine	3
	Othello	3
	Pacman	5
	Snake	4
	Solitaire	2

Table 3
Descriptive statistics of the application sample.

Parameter	Minimum	Median	Maximum	Mean
Age (years)	<1	4	8	4
Size (binary length)	209	11,349	556,697	61,122
Size (methods)	14	197	10,110	1610

All statistical analyses have been performed using SPSS 15 on the sample of 63 projects. In order to test our hypotheses, we used ordinary least squares to estimate our regression model.

5.3. Results of statistical analyses

Table 6 shows the coefficients of the regression model represented by Eq. (4). Table 7 reports the summary variables of the regression models. The *R*-square value (0.484) is strong, and indicates that the fit of the model to data is quite good. Coefficient *b*₃, representing the interaction effect between framework entropy and size on energy efficiency, is highly significant, suggesting that (as hypothesized in H2), there is an interaction between application size and framework entropy.

In order to have a better understanding of the main effect of framework entropy on energy efficiency, suggested by hypothesis H1, and its interaction effect with size, suggested by hypothesis H2, we differentiated expression (4) with respect to *ENTROPY* and analyzed how *EE* varies as a function of *SMALL* and *ENTROPY*. The results are shown in Table 8 and plotted in Fig. 3.¹

Table 8 shows the value of *EE* predicted by the model for different values of *SMALL* (i.e., size) and *ENTROPY* (i.e., use of application development environments). The medium values correspond to the mean values of size and framework entropy in the sample. Low and high framework entropy are computed for the values of *ENTROPY* corresponding to the mean plus and minus the standard deviation of the variable in the sample, respectively. Small and large size values are computed respectively for the values 1 and 0 of the variable *SMALL* that distinguishes the two clusters.

These results show that the main effect of framework entropy on energy efficiency is positive, i.e. the use of frameworks and external libraries is beneficial to energy efficiency for small to

Table 5
Mean and standard deviation of variables.

Variable	Mean	Standard deviation
EE	.639	.292
ENTROPY	.0015	.003
SMALL	.381	.490
AGE	3.980	2.255
FTP	.170	.383
CALC	.160	.368
CALENDAR	.080	.272
IMAGE	.110	.317
TEXT	.060	.246
WEB	.060	.246
ERP	.030	.177
GAME	.320	.469

medium applications. This is reflected in Fig. 3, where the slopes of the lines for small and medium applications are both positive, indicating that as framework entropy increases, so does energy efficiency. However, as Fig. 3 also illustrates, for large applications, the slope of the line is negative, indicating that as framework entropy increases, energy efficiency decreases.

According to these results, hypothesis H1 (framework entropy has a detrimental effect on energy efficiency on average) is not verified, whereas hypothesis H2 (the detrimental effect of framework entropy on energy efficiency is more intense for larger applications) is verified. Interestingly, we find that framework entropy has a mixed effect on energy efficiency whereby greater framework entropy is helpful (in terms of improving energy efficiency) for small to medium applications, but, as anticipated, for larger applications, greater framework entropy reduces energy efficiency. We discuss potential explanations for these findings in our Discussion section.

Table 6 also suggests that different functional types have different levels of energy efficiency. In order to investigate this effect we used the estimated regression coefficients to compute the effect of each functional type binary variable on the average values of *EE*. Results are shown in Table 9 and Fig. 4.

As reflected by Table 9 and Fig. 4, text editors, ERPs and image editors are the least energy efficient applications, whereas calendars are the most energy efficient. More generally, the application types that appear to be less energy efficient are those that present higher degrees of functional complexity, such as ERPs, editors and games, which need to take into account and process several types of input data almost in real time.

6. Discussion

The objective of this study was to examine the relationship between the use of traditional application development environments and the energy efficiency of application software, starting from the observation that software plays a critical role in the overall energy consumption of IT. This research has focused on user-oriented applications, which have been the main target of application development environments [18].

Energy efficiency is a new dimension of software quality that is assuming more and more importance for its industrial implications. The ISO 9126:2003 and 25000:2005 software metrics are organized along the following dimensions: (1) functionality, (2) reliability, (3) usability, (4) efficiency, (5) maintainability, and (6) portability. None of the ISO software quality dimensions seems conceptually related to energy efficiency. Therefore, assessing energy efficiency requires new software quality metrics that are currently lacking. Our work proposes a methodology to empirically measure the energy efficiency induced by software applications and a new metric to evaluate energy efficiency. Energy efficiency metrics are essential in order to compare different applications

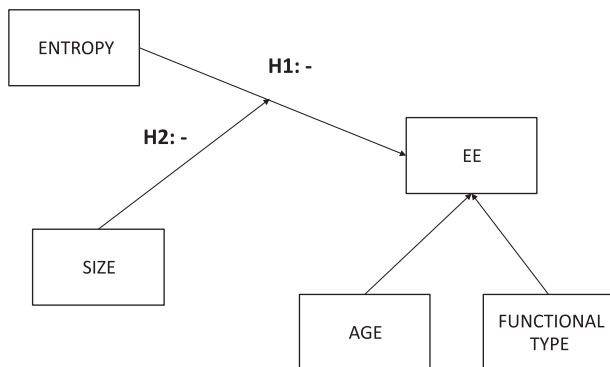
¹ The values of *EE* vary from 0 to 1 and the values of *ENTROPY* are greater or equal to 0 according to their definition. However, in this simulation these boundaries condition are not respected for plotting reason.

Table 4
Correlation table.

Variable	EE	ENTROPY	SMALL	AGE	FTP	CALC	CALENDAR	IMAGE	TEXT	WEB	ERP	GAME
EE	1											
ENTROPY	-.158	1										
SMALL	-.108	-.132	1									
AGE	.065	-.056	-.292*	1								
FTP	.131	-.163	-.103	.256	1							
CALC	.316*	-.074	-.072	.130	-.200	1						
CALENDAR	.237	-.040	.132	-.297*	-.135	-.128	1					
IMAGE	-.235	-.141	.035	-.302*	-.163	-.154	-.104	1				
TEXT	-.379**	.521**	-.070	.032	-.120	-.113	-.076	-.092	1			
WEB	-.049	-.084	.062	.174	-.120	-.013	-.076	-.092	-.068	1		
ERP	-.196	-.079	-.142	-.011	-.083	-.079	-.053	-.064	-.047	-.047	1	
GAME	-.035	.111	.097	-.010	-.314**	-.296*	-.200	-.241	-.178	-.178	-.123	1

* Correlation is significant at the 0.05 level (2-tailed).

** Correlation is significant at the 0.01 level (2-tailed).

**Fig. 2.** Statistical research model.

and to provide managers with adequate tools to support their acquisition choices or to correctly supervise software development processes. Energy efficiency metrics are quite different from energy consumption metrics, as they must normalize the amount of energy consumed by a measure of the work performed. Whereas there are a number of efficiency metrics for hardware (e.g., watt per tpm, watt per mips, watt per flop, etc.) none have been defined for software, as workloads are complex and difficult to quantify. Our energy efficiency metric allows us to compare the energy efficiency of applications belonging to different functional areas, even though it is based on the execution of a specific benchmark workload. Further research should aim at refining this methodology and identifying new software energy efficiency metrics.

As previously discussed, traditional application development environments are founded on the componentization and automation principles, and widely rely on the use of frameworks, libraries,

Table 6
Coefficients of the regression model with dependent variable EE.

	Unstandardized coefficients		Standardized coefficients	T	P-value
	B	Std. error			
ENTROPY (b1)	-6.912	11.967	-.080	-.578	.566
SMALL (b2)	-.233	.086	-.381	-2.695	.010
SMALL-ENTROPY (b3)	142.674	49.730	.405	2.869	.006
AGE (b4)	.001	.018	.009	.068	.946
FTP (b5)	.093	.106	.115	.878	.385
CALC (b6)	.238	.105	.291	2.364	.023
CALENDAR (b7)	.323	.129	.309	2.502	.016
IMAGE (b8)	-.108	.117	-.120	-.923	.361
TEXT (b9)	-.311	.156	-.268	-1.995	.052
WEB (b10)	.059	.156	.044	.376	.709
ERP (b11)	-.311	.186	-.193	1.668	.102
Constant (e)	.632	.107		5.888	.000

Table 7
Summary variables of the regression model.

Variable	Value
R	.696
R ²	.484
Adjusted R ²	.355
Standard error of the estimate	.2419

Table 8

Energy efficiency for different sizes of applications and different levels of framework entropy.

Size	Entropy		
	Low	Medium	High
Small	.194704	.650875	1.107046
Medium	.503237	.662630	.822023
Large	.693103	.669864	.646625

and abstraction layers in order to increase development efficiency. Our premise was that while use of these environments may facilitate efficiency in coding, they do not necessarily lead to greater energy efficiency. In testing our hypotheses, we have used a new metric, called framework entropy, measuring to what extent an application uses external libraries and frameworks rather than code developed from scratch.

Our results show that for small to average size applications the use of application development environments is actually associated with greater software energy efficiency, but that for larger applications the opposite is true. This disconfirms Hypothesis H1, but supports H2. From a managerial point of view, this result indicates that traditional application development environments, designed to optimize development efficiency, positively impact the energy efficiency of smaller applications. A possible explanation for this finding is that the average programmer who develops new functions from scratch is unlikely to be able to “beat” the energy efficiency of standard components. In summary, the use of traditional application development environments for smaller programs may be *de facto* beneficial not only to reduce development costs, but also to improve energy efficiency.

However, framework entropy is found to have a detrimental effect on energy efficiency as soon as the size of applications grows. Results indicate that for larger applications the effect of greater framework entropy on energy efficiency is negative. A possible explanation is that the growth of a large application typically occurs by embedding initial modules inside other larger modules. This may unnecessarily increase the number of layers that must be crossed to execute a single operation. Overall, our results suggest that there is a trade-off between development efficiency and

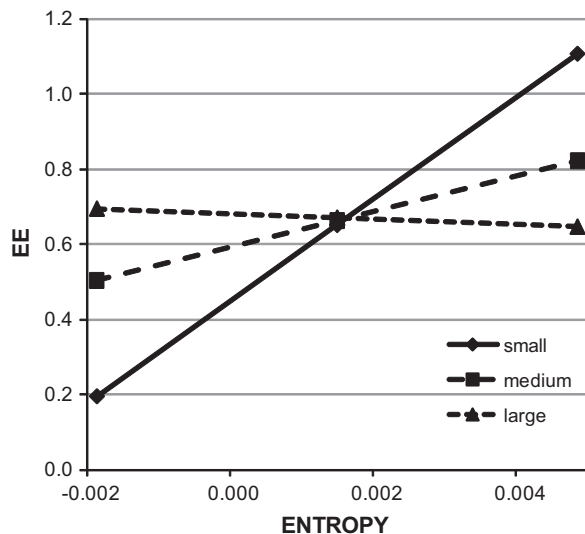


Fig. 3. Simulation of the model to analyze the interaction effect on energy efficiency of framework entropy with size.

Table 9
Effect of functional type on energy efficiency.

Functional type	Effect on average EE
TEXT_EDIT	.308453
ERP	.308732
IMAGE_EDIT	.511010
GAME	.619239
WEB	.677822
FTP	.712511
CALC	.867131
CALENDAR	.942454

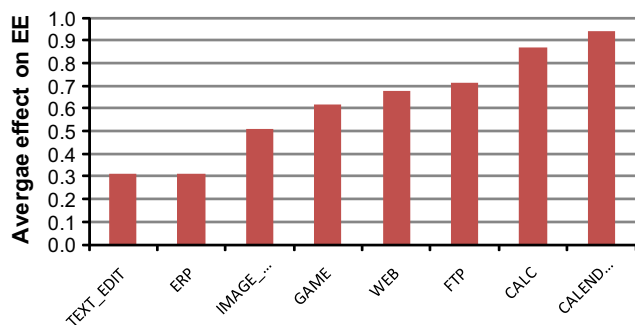


Fig. 4. Effect of functional type on energy efficiency.

energy efficiency that depends on the size of the application and the extent of use of development environments.

It should also be noted that the applications in our sample are relatively small compared to traditional MIS applications used in corporate environments. The results shown in Table 8 and Fig. 3 clearly show that size has an impact on the relationship between framework entropy and energy efficiency, whereby greater framework entropy negatively impacts energy efficiency for larger applications. For most MIS applications (which are typically larger than those in our sample) the use of frameworks, external libraries, and abstraction layers, is therefore likely to have a detrimental impact on energy efficiency. This may have a significant impact on the total cost of ownership for MIS applications, as the development and customization of MIS applications usually greatly relies on traditional development environments [4]. Future research should investigate the relationship between framework entropy and energy efficiency for very large applications.

To provide deeper insight into why the use of frameworks may have a detrimental effect on energy efficiency, let us consider in detail two different open source ERPs, Adempiere and OpenBravo, running on the same database system and hardware infrastructure, and with the same parameter setting. As we observed in Section 3, one of the main differences between the two ERPs is that OpenBravo uses HIBERNATE (an Object Relational Mapping framework) to access the database, whereas Adempiere directly codes SQLj instructions into its classes. Of course it is much easier to develop, modify and customize a class that uses HIBERNATE rather than SQLj, as no low level information on the structure of the application and no advanced database programming skills are required. However, the execution of the framework poses an overhead for the processor executing the system and requires much more power than the execution of SQL instructions. We tested the two ERPs while executing a common benchmark workload (insertion of a new product) and analyzed the code to find out the power required by each portion of the program. Fig. 5 represents the total power consumed by the server while running the two ERPs performing the same operation. HIBERNATE has a very limited impact on time performance (+5%) compared to the SQLj instructions, but a large impact on the energy consumed (+50%).

Our results also suggest that different functional types of applications have different levels of energy efficiency. In particular (see Fig. 4), ERPs, text and image editors and games are less energy efficient than applications such as FTP clients and servers, and calendars. Games and image editors are characterized by a very intense use of the processor. ERPs and text editors usually have advanced and complex user interfaces that provide input data to be processed almost in real time. In particular, ERPs manage complex data structures that, as discussed before, require additional layers in the structure of the application. Conversely, calendars and similar applications mainly focus on the memorization of data and only perform very simple processing operations. FTP clients and servers are mostly transactional applications, with very simple interfaces. This suggests that the functional complexity of an application, and in particular the extent to which its execution is processor intensive, negatively impacts its energy efficiency. This may be interpreted on the basis of the empirical results cited in Section 2.1, and which we presented in a previous paper [12], on the different consumption of processors and storage devices. The power absorbed by the processor is directly related to its usage, whereas the power absorbed by dynamic RAM and traditional hard disk devices is mainly independent of usage. Accordingly, the applications that make a more intense use of the processor rather than memory tend to be less energy efficient.

This suggests new research lines to be investigated in order to make software more energy efficient: if an application somehow shifts its computational workload from the processor to the storage

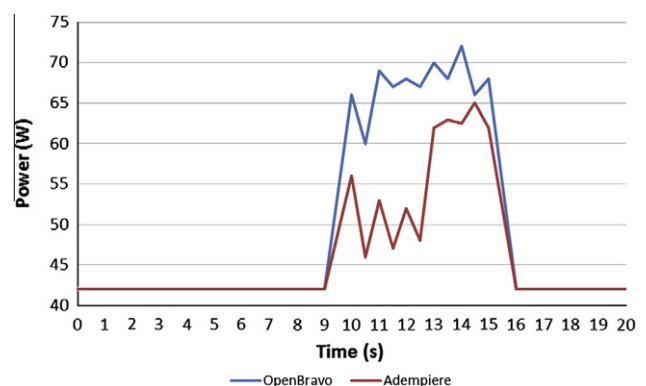


Fig. 5. Power consumption of two different ERPs executing the same operation on the same hardware infrastructure.

it may achieve higher energy efficiency, because the power consumption of RAM and disks is like a “sunk cost”. Memoization techniques (i.e., elaboration techniques that return output results by caching the results of previous computations) [33,29], optimized use of garbage collection in Java, and in general optimized use of memory should be investigated as potential strategies to increase software energy efficiency.

Overall, our results suggest a number of questions for further research. Can highly skilled developers who can develop efficient code from scratch, beat the performance of standard routines from the perspective of energy efficiency? If so, at what cost? Is energy efficiency economically beneficial for large applications? Alternatively, what are the cost trade-offs between maintainability and energy efficiency? What are the software metrics that should be used to assess these trade-offs?

7. Conclusion, limitations and future work

The results from our study suggest that development practices based on the intensive use of traditional development environments are beneficial to the energy efficiency of small applications, but may be detrimental to the energy efficiency of larger applications.

Our research contributes by introducing a measure of software energy efficiency. To the best of our knowledge this is the first measure of its kind for high level software applications. Second, we have related the use of software application development environments to software energy efficiency suggesting that there may be a trade off between development efficiency and energy efficiency.

Our research is subject to some limitations that can be addressed in future work. First, our study is limited to open source applications as we needed access to the code to measure size and framework entropy. Open source applications are different from those of traditional software applications [35,9]. They are characterized by specific development practices and, on average, tend to be smaller than traditional MIS applications developed in

corporate contexts. Further work will extend this research to very large and traditional closed-source applications, provided that we find a suitable source of applications that allows access to code. However, many firms are participating in open source projects: a recent stream of literature ([8,10]) has shown that open source projects are widely participated in by firms. In particular 31% of SourceForge projects are participated in by firms, which on average contribute more than half of the code to these projects. As our research analyses focus on software code, we believe that our sample provides a significant testbed for verifying our hypotheses.

Second, the metric of framework entropy adopted in this study may be related to the experience of the developers involved in development projects. Also programming style influences the metric, in particular the developer’s ability or habit of employing the whole language syntax. Even though a manual inspection of some applications confirmed that our interpretation of the metric is correct (see Section 4.2), it would be interesting to analyze the use of application development environments directly to provide further evidence of their impact on software energy efficiency.

Third, our experiments have been performed on a lower-end server. The progress of hardware technology combined with the adoption of higher-end infrastructures may affect our results, which will need to be verified on these systems.

Acknowledgments

We thank Giulia Formenti, Stefano Gallazzi, and Gabriele Galli, former students, for their substantial contribution to this work. We thank Francesco Merlo for his invaluable suggestions in the early phases of the research. This research project has been partially supported by Accenture Italia S.p.A.

Appendix A. Applications sample

Category	Application	URL
Browser	jBrowser	http://code.google.com/p/jbrowser/
Browser	JXWB	http://sourceforge.net/projects/jxwb/
Calculator	CIAC	http://one.abstractions.me/ciac-ciac-is-another-calculator/
Calculator	Fir4j	http://fir4j.sourceforge.net/
Calculator	JAC	http://jac-java-calculator.downloadaces.com/
Calculator	jCalc	http://sourceforge.net/projects/javacalc/
Calculator	ScientificCalculator	http://jsccalc.sourceforge.net/
Calculator	SolCalc	http://sourceforge.net/projects/solcalc/
Calendar	Kalender	http://118.98.171.130/Source%20Code/Kalender/
Calendar	MyTaskScheduler	http://sourceforge.net/projects/mytaskscheduler/
Calendar	Panda	http://sourceforge.net/projects/hgpanda/
Calendar	Pcalendar	http://sourceforge.net/projects/php-calendar/
Calendar	Remider	http://neon-reminder.software.informer.com/
ERP	Adempiere	http://adempiere.org/home/
ERP	OpenBravo	http://www.openbravo.com/
FTP client	GSFTP	http://ostatic.com/gssftp
FTP client	jsFTP	http://gitorious.org/jsftp-server
FTP client	jsvFTP	http://jvftp.sourceforge.net/
FTP client	VirgoFTP	http://sourceforge.net/projects/qftp/
FTP client	YAFTP	http://yaftp.sourceforge.net/
FTP Server	Anomic	http://www.anomic.de/AnomicFTPServer/Download.htm
FTP server	danoFTP	http://sourceforge.net/projects/danoftp/
FTP Server	Jupiter	http://sourceforge.net/projects/jupiter-ftp/
FTP Server	xjFTP	http://sourceforge.net/projects/xjftp/
Game – Pacman	Dracman	http://dracman.sourceforge.net/

(continued on next page)

Appendix A (continued)

Category	Application	URL
Game – Pacman	Packman	http://packman.sourceforge.net/
Game – Pacman	Pelletquest	http://sourceforge.net/projects/pelletquest/
Game – Pacman	Phoenix	http://sourceforge.net/project/shownotes.php?group_id=28605&release_id=38003
Game – Pacman	SimpleJ Pacman	http://www.simplej.org/
Game – Arkanoid	Jarky	http://sourceforge.net/projects/jarky/
Game – Arkanoid	OpenArkanoid	http://sourceforge.net/projects/openarkanoid/
Game – Backgammon	jBackgammon	http://sourceforge.net/projects/jbackgammon/develop
Game – Backgammon	JGammon	http://jgam.sourceforge.net/
Game – Backgammon	Stonesthrow	http://download.cnet.com/Stones-Throw/3000-18522_4-9300.html
Game – Mine	jExplosion	http://sourceforge.net/projects/jexplosion/
Game – Mine	RoxMine	http://sourceforge.net/projects/roxminesweeper/develop
Game – Mine	jSaper	http://sourceforge.net/projects/jsaper/
Game – Othello	Billy	http://sourceforge.net/projects/billy/
Game – Othello	Jmorpheus	http://sourceforge.net/projects/jmorpheus/
Game – Othello	Reversi	http://code.google.com/p/othello-reversi/updates/list
Game – Snake	Jsnake	http://sourceforge.net/projects/jsnake/
Game – Snake	Kurvetest	http://zh.sourceforge.jp/projects/sfnet_kurve-online/
Game – Snake	Snaax	http://ostatic.com/snaax/home/1
Game – Snake	SnakeVSSnake	http://snakevssnake.sourceforge.net/
Game – Solitaire	CalorSolitaire	http://sourceforge.net/projects/solitaire/
Game – Solitaire	JavaSolitaire	http://sourceforge.net/projects/javaklondike/
Image editing	Jmjrst	http://jmjrst.sourceforge.net/
Image editing	Jrezz	http://sourceforge.net/projects/jrezz/
Image editing	JScale	http://j-scale.sourceforge.net/
Mail server	Hermes	http://hermes.mozdev.org/index.html
Mail server	James	http://james.apache.org/download.cgi
Mail server	JES	http://javaemailserver.sourceforge.net/index.html
PDF merger	jpdf tweak	http://jpdf tweak.sourceforge.net/
PDF merger	PDFsam	http://www.pdfsam.org/
Spreadsheet	Bsheet	http://bsheet.sourceforge.net/
Spreadsheet	CleanSheets	http://csheets.sourceforge.net/
Spreadsheet	Jeppers	http://sourceforge.net/projects/jeppers/
Spreadsheet	SharpTools	http://sourceforge.net/projects/sharptools/
Spreadsheet	XSheet	http://code.google.com/p/xsheet/
Text editor	JavaTextEditor	http://sourceforge.net/projects/javatexteditor/
Text editor	POM	http://sourceforge.net/projects/nrmpom/
Text editor	Rtext	http://sourceforge.net/projects/rtext/
Text editor	TextTrix	http://sourceforge.net/projects/texttrix/

References

- [1] ACEEE, A Smarter Shade of Green, ACEEE Report for the Technology CEO Council, 2008.
- [2] S. Albers, H. Fujiwara, Energy-efficient algorithms for flow time minimization, *ACM Transactions on Algorithms* (TALG) 3 (49) (2007).
- [3] D. Ardagna, C. Francalanci, A cost-oriented methodology for the design of web based IT architectures, in: *ACM Symposium on Applied Computing*, 2002, pp. 1127–1133.
- [4] R.D. Banker, G.B. Davis, S.A. Slaughter, Software development practices, software complexity, and software maintenance performance: a field study, *Management Science* 44 (4) (1998) 433–450.
- [5] L. Benini, G. De Micheli, System-level power optimization: techniques and tools, *ACM Transaction on Design Automation of Electronic Systems* 5 (2) (2000) 115–192.
- [6] R. Bianchini, R. Rajamony, Power and energy management for server systems, *Computer* 37 (11) (2004) 68–74.
- [7] F.P. Brooks, No silver bullet essence and accidents of software engineering, *Computer* 20 (4) (1987) 10–19.
- [8] E. Capra, C. Francalanci, F. Merlo, Software design quality and development effort: an empirical study on the role of governance in Open Source projects, *IEEE Transaction on Software Engineering* 34 (6) (2008) 765–782.
- [9] E. Capra, A.I. Wasserman, A framework for evaluating managerial styles in Open Source projects, in: *Open Source Systems Conference*, 2008, pp. 1–14.
- [10] E. Capra, C. Francalanci, F. Merlo, C. Rossi Lamastra, A survey on firms' participation in Open Source Community projects, in: *Open Source Systems Conference*, 2009, pp. 225–236.
- [11] E. Capra, F. Merlo, Green IT: everything starts from the software, in: *European Conference of Information Systems*, 2009.
- [12] E. Capra, G. Formenti, C. Francalanci, S. Gallazzi, The impact of MIS software on IT energy consumption, in: *European Conference of Information Systems*, 2010.
- [13] A. Chatzigeorgiou, G. Stephanides, Energy metric for software systems, *Software Quality Journal* 10 (2002) 335–371.
- [14] W. Fornaciari, P. Gubian, D. Sciuto, C. Silvano, Power estimation of embedded systems: a hardware/software codesign approach, *IEEE Transaction on VLSI Systems* 6 (2) (1998) 266–275.
- [15] M. Fowler, A survey of object oriented analysis and design methods, in: *Proc. Of Int'l Conf. on Software Engineering*, 1997.
- [16] M. Fowler, *Refactoring: Improving the Design of Existing Code*, Addison-Wesley Professional, 1999.
- [17] E. Gamma, R. Helm, R. Johnson, J. Vlissides, *Design Patterns: Element of Reusable Object-oriented Software*, Addison-Wesley, Reading, Massachusetts, 1994.
- [18] J. Greenfield, K. Short, Software factories: assembling applications with patterns, models, frameworks and tools, in: *18th ACM SIGPLAN Conference on Object-oriented Programming, Systems, Languages, and Applications*, 2003, pp. 16–27.
- [19] J. Howison, K. Crowston, The perils and pitfalls of mining SourceForge, in: *Proc. Int'l Workshop on Mining Software Repositories*, 2004, pp. 7–12.
- [20] P. Huang, S. Ghiasi, Efficient and scalable compiler-directed energy optimization for realtime applications, *ACM Transactions on Design Automation of Electronic Systems* 12 (3) (2007) 1–16.
- [21] ISO/IEC, TR 9126:2003, *Software Engineering – Product Quality*, International Organization for Standardization, Geneva, Switzerland.
- [22] ISO/IEC, TR 25000:2005, *Software Engineering – Software Product Quality Requirements and Evaluation (SQuaRE)*, International Organization for Standardization, Geneva, Switzerland.
- [23] S.L. Josselyn, B. Dillon, M. Nakamura, R. Arora, S. Lorenz, T. Meyer, R. Maceska, L. Fernandez, "Worldwide and Regional Server 2006–2010 Forecast", IDC Report, November 2006.

- [24] G. Kaefler, J. Haid, G. Schall, R. Weiss, The standard power estimation interface for software components, in: *International Symposium on Wearable Computers*, 2001.
- [25] N. Kandasamy, S. Abdelwahed, J. Hayes, Self-optimization in computer systems via on-line control: application to power management, in: *Proc. Of Int'l Conf. on Autonomic Computing*, 2004, pp. 54–61.
- [26] W. Kozaczynski, G. Booch, Component-based software engineering, *IEEE Software*, Sept-Opt. 1998, 1998, pp. 34–36.
- [27] R. Kumar, Important Power, Cooling and Green IT Concerns, *Gartner Report*, January 2007.
- [28] H. Liu, M. Parashar, S. Hariri, A component-based programming model for autonomic applications, in: *Proc. of Int'l Conf. on Autonomic Computing*, 2004, pp. 10–17.
- [29] A. Ma, M. Zhang, K. Asanovic, Way memoization to reduce fetch energy in instruction cache, *Workshop on Complexity-Effective Design*, 28th ISCA, Gothenburg, Sweden, 2001.
- [30] C.C. Mann, Why is software so bad, *Technology review* 105 (6) (2002).
- [31] P. Marttiin, M. Rossi, V.-P. Tahvainanen, K. Lyytinen, A comparative review of CASE shells—a preliminary framework and research outcomes, in: *Information and Management*, vol 25, no. 1, 2nd ed., 1993, pp. 11–31.
- [32] J.D. Meier, S. Vasireddy, A. Babbar, A. Mackman, Improving .NET Application Performance and Scalability, *Microsoft Corp.*, 2004.
- [33] D. Michie, Memo functions and machine learning, *Nature* 218 (1968) 19–22.
- [34] J.M. Rabaey, M. Pedram, *Low Power Design Methodologies*, Kluwer Academic Pub., 1996.
- [35] E.S. Raymond, *The Cathedral and the Bazaar*, O'Reilly, Cambridge, Mass., 1999.
- [36] C. Seo, S. Malek, N. Medvidovic, Component-level energy consumption estimation for distributed java-based software systems, in: *International Symposium on Component Based Software Engineering (CBSE 2008)*, Karlsruhe, Germany, October 2008.
- [37] C. Seo, G. Edwards, D. Popescu, S. Malek, N. Medvidovic, A Framework for estimating the energy consumption induced by a distributed system's architectural style, *ESEC/FSE Workshop on Specification and Verification of Component-Based Systems (SAVCBS 2009)*, Amsterdam, Netherlands, August 2009.
- [38] C.E. Shannon, Prediction and entropy of printed English, *The Bell System Technical Journal* 30 (1951) 50–64.
- [39] A.C. Shaw, Reasoning about time in higher-level language software, *IEEE Transactions on Software Engineering* 15 (7) (1989) 875–889.
- [40] M. Sitaraman, G. Kulczycki, J. Krone, W.F. Ogden, A.L.N. Reddy, Performance specifications of software components, in: *Proc. of the 2001 Symposium on Software Reusability*, 2001, pp. 3–10.
- [41] A. Sivasubramaniam, M. Kandemir, N. Vijaykrishnan, M.J. Irwin, Designing energy-efficient software, in: *International Parallel and Distributed Processing Symposium (IPDPS)*, vol. 2, 2002, pp. 176.
- [42] E. Stanford, Environmental trends and opportunity for computer system power delivery, in: *20th Int'l Symposium on Power Semiconductor Devices and IC's*, 2008.
- [43] B. Steffen, P. Narayan, Full life-cycle support for end-to-end processes, *Computer* 40 (11) (2007) 64–73.
- [44] C. Szyperski, J. Bosch, W. Weck, *Component Oriented Programming*, Springer, 1999.
- [45] E.J. Weyuker, F.J. Vokolos, Experience with performance testing of software systems: issues, an approach, and case study, *IEEE Transactions on Software Engineering* 26 (12) (2000) 1147–1156.
- [46] Y. Zhu, G. Magklis, M.L. Scott, C. Ding, D.H. Albonesi, The energy impact of aggressive loop fusion, in: *Proceedings of the 13th international Conference on Parallel Architectures and Compilation Techniques*, 2004.